DocVision

Document Intelligence Project — Workflow, Architecture & Development Phases

1. Project Objective

DocVision is an end-to-end document-understanding system. The goal is to accept a document image, understand its contents using a deep-learning document AI model, and return structured information as JSON.

Core Pipeline

```
Document Image → Image Preprocessing → Document AI Model → Structured Output Parser → JSON Response
```

Example Target Output

```
{
  "items": [
    {
      "name": "Nasi Campur Bali",
      "quantity": 1,
      "price": 75000
    }
  ],
  "total": 75000
}
```

2. Overall System Architecture

Layer	Component	Purpose
Client	User / Application	Uploads a document image
API	FastAPI REST API	Receives the document and returns results
Preprocessing	Image Processor	RGB conversion, resizing, normalization as required
Model	Document AI / Vision-Language Model	Understands the document and generates structured information
Post-processing	Output Parser / Validator	Converts model output into clean, validated JSON
Response	JSON	Returns structured document information to the client

Architecture flow: User/Client → FastAPI → Image Preprocessing → Document AI Model → Post-processing → Structured JSON

3. Development Workflow

Phase	Stage	Primary Goal
1	CORD Exploration	Understand images, annotations, fields, and data distribution
2	Annotation Preprocessing	Parse, clean, normalize, and generate model targets
3	Image Preprocessing	Prepare images consistently for the selected model
4	Model Selection & Baseline	Select a feasible document AI approach and establish a baseline
5	Fine-tuning / Training	Train or fine-tune the model on CORD

Phase	Stage	Primary Goal
6	Evaluation	Measure extraction and document-understanding performance
7	Inference Pipeline	Build prediction and post-processing for unseen documents
8	FastAPI & Deployment	Expose the trained system as a production-style API

Phase 1 — CORD Exploration

• Load CORD-v2 and verify train/validation/test splits.

• Inspect receipt images and raw ground-truth annotations.

• Understand the gt_parse hierarchy and available fields.

• Measure field frequency, menu sizes, and missing fields.

• Notebook: notebooks/01_cord_exploration.ipynb.

• Current status: IN PROGRESS.

Phase 2 — Annotation Preprocessing

• Parse JSON annotations.

• Normalize fields such as item name, quantity, price, subtotal, and total.

• Convert values such as '1 x' $\rightarrow$ 1 and '75,000' $\rightarrow$ 75000.

• Handle missing or malformed annotations.

• Generate clean model targets.

• Reusable script: scripts/prepare_cord.py.

Phase 3 — Image Preprocessing

• Determine model-specific image requirements.

• Convert images to the required format/color space.

• Resize and normalize according to the selected model.

• Keep preprocessing identical between training and inference.

Phase 4 — Model Selection & Baseline

• Compare a practical OCR-plus-NLP baseline with a document AI / vision-language approach.

• Select a model compatible with the available hardware and project scope.

• Establish a baseline before fine-tuning.

• Document the rationale for the chosen architecture.

Phase 5 — Fine-tuning / Training

• Use the 800 training samples for training.

• Use the 100 validation samples for model selection and tuning.

• Keep the 100 test samples untouched until final evaluation.

• Track training loss and validation performance.

• Save the trained model/checkpoints.

Phase 6 — Evaluation

• Evaluate predictions against ground truth.

• Measure field-level extraction performance where appropriate.

• Measure structured-output correctness / exact matching where appropriate.

• Use OCR/text metrics only when they match the selected output representation.

• Analyze failure cases and common extraction errors.

Phase 7 — Inference Pipeline

• Accept a previously unseen document image.

• Apply the same preprocessing used during training.

• Run the trained model.

• Parse and validate the model output.

• Return clean structured JSON.

Phase 8 — FastAPI & Deployment

• Create FastAPI endpoints for document upload and prediction.

• Define request/response schemas with Pydantic.

• Load the trained model once at application startup.

• Add error handling and validation.

• Containerize with Docker.

• Test the API with real unseen receipts.

4. Dataset Structure

CORD-v2 currently provides 1,000 receipt images: 800 training, 100 validation, and 100 test examples.

Each sample contains an image and ground_truth. The ground truth contains a gt_parse object, which can include fields such as menu and other receipt-level information.

```
Typical hierarchy: ground_truth → gt_parse → menu → individual item → nm / cnt / price.
```

5. Recommended Project Structure

```
DocVision/
■■■ data/
■   ■■■ raw/
■   ■■■ processed/
■   ■■■ outputs/
■■■ notebooks/
■   ■■■ 01_cord_exploration.ipynb
■■■ scripts/
■   ■■■ prepare_cord.py
■   ■■■ train.py
■   ■■■ evaluate.py
■■■ src/
■   ■■■ preprocessing/
■   ■   ■■■ image_processor.py
■   ■■■ models/
■   ■   ■■■ document_model.py
■   ■■■ inference/
■   ■   ■■■ predictor.py
■   ■■■ utils/
■       ■■■ postprocessing.py
■■■ api/
■   ■■■ main.py
■   ■■■ schemas.py
■   ■■■ routes.py
■■■ models/
■   ■■■ docvision_model/
■■■ tests/
■   ■■■ test_preprocessing.py
■   ■■■ test_inference.py
■   ■■■ test_api.py
■■■ requirements.txt
■■■ Dockerfile
■■■ README.md
```

6. Current Progress

Task	Status
CORD-v2 download/load	Complete
Dataset split verification	Complete
Image inspection	Complete
Ground-truth inspection	Complete
Python dictionary / JSON concepts	Complete
Field-frequency analysis	In progress
Annotation normalization	Next
Image preprocessing	Pending
Model selection	Pending
Training/fine-tuning	Pending
Evaluation	Pending
Inference	Pending

Task	Status
FastAPI	Pending
Docker / deployment	Pending

7. Final End-to-End Flow

```
CORD Exploration → Data Cleaning → Annotation Normalization → Image Preprocessing → Model Selection → Baseline → Fine-tuning → Evaluation → Inference → FastAPI → Docker
```

Key principle: Do not jump directly from dataset loading to training. The project should progress through understanding, cleaning, normalization, baseline establishment, training, evaluation, inference, and deployment so every component can be explained in an interview.